◉ 11-C 計算 Linear Convolution

We know that when $y[n] = x[n] * h[n] = \sum_k x[n-k]h[k]$

then $y[n] = IFFT\left(FFT\{x[n]\}\,FFT\{h[n]\}\right)$

(*N* points) (*M* points)

(*P* points)

$$\boxed{P \geq M + N - 1}$$

convolution output: $M+N-1$ points

① ex: $N=512$, $M=21$
$P \geq 532 \Rightarrow$ choose $P=560$

② ex: $N=100$, $M=19$
$P \geq 118 \Rightarrow$ set $P=120$

But how do we implement it correctly?

How do we choose *P* ?

2 P-point FFTs + P-point multiplication

①乘法量: $2\,MUL_{560} + 560 \times 3 = 2 \times 3100 + 1680 = 7880$

實數乘法量

Note: When $y_1[n] = IFFT_P\left(FFT_P\{x_1[n]\}\,FFT_P\{h_1[n]\}\right)$

then $y_1[n] = \sum_{k=0}^{P-1} x_1[((n-k))_P]h_1[k]$ (circular convolution)

FFT_P: the *P*-point FFT $IFFT_P$: the *P*-point inverse FFT

$((a))_P$: *a* 除以 *P* 的餘數

Linear convolution $y[n]=\sum_{k=0}^{M-1}x[n-k]h[k]$

Circular convolution $y_1[n]=\sum_{k=0}^{P-1}x_1[((n-k))_P]h_1[k]$

$$\boxed{x_1[n]=x[n] \quad for\ 0\le n<N, \quad x_1[n]=0 \quad for\ N\le n\le P-1}$$

$$\boxed{h_1[n]=h[n] \quad for\ 0\le n<M, \quad h_1[n]=0 \quad for\ M\le n<P-1}$$

When $P\ge M+N-1$

$$y[n]=y_1[n] \quad for\ 0\le n<N+M-2,$$
$$y[n]=0 \quad for\ N+M-1\le n<P-1$$

When $y_1[n] = IFFT_P\left(FFT_P\{x_1[n]\}\, FFT_P\{h_1[n]\}\right)$

then $y_1[n] = \sum_{k=0}^{P-1} x_1[((n-k))_P]h_1[k]$

(Proof) Suppose that $X_1[m] = FFT_P\{x_1[n]\}$, $H_1[m] = FFT_P\{h_1[n]\}$

$$\frac{1}{P}\sum_{m=0}^{P-1} X_1[m]H_1[m]e^{j\frac{2\pi m}{P}n} = \frac{1}{P}\sum_{m=0}^{P-1}\sum_{k=0}^{P-1} x_1[k]e^{-j\frac{2\pi m}{P}k}\sum_{s=0}^{P-1} h_1[s]e^{-j\frac{2\pi m}{P}s}e^{j\frac{2\pi m}{P}n}$$

$$= \frac{1}{P}\sum_{k=0}^{P-1}\sum_{s=0}^{P-1} x_1[k]h_1[s]\sum_{m=0}^{P-1} e^{j\frac{2\pi(n-s-k)}{P}m}$$

$$= \frac{1}{P}\sum_{k=0}^{P-1}\sum_{s=0}^{P-1} g[k]h[s]P\delta_d[((n-s-k))_P]$$

$$= \sum_{k=0}^{P-1} g[k]h[((n-k))_P]$$

Here we apply $\sum_{n=0}^{P-1} e^{j\frac{2\pi a}{P}n} = P\delta_d[((a))_P]$ $((a))_P$: the remainder of a after divided by P

[Discrete Circular Convolution and Discrete Linear Convolution]

A discrete linear time-invariant (LTI) system can always be expressed a discrete linear convolution:

$$y[n] = x[n] * h[n] = \sum_{k=0}^{N-1} x[k]h[n-k]$$

However, the convolution implemented by the DFT is the discrete circular convolution:

If

$$y_1[n] = IDFT\left(DFT\{x_1[n]\}\, DFT\{h_1[n]\}\right) = IDFT\left(X_1[m]H_1[m]\right)$$

then

$$y_1[n] = x_1[n] *_c h_1[n] = \sum_{k=0}^{P-1} x_1[k]h_1[((n-k))_P]$$

$((a))_P$: the remainder of a after divided by P

linear convolution: $$y[n]=x[n]*h[n]=\sum_{k=0}^{N-1}x[k]h[n-k]$$

circular convolution: $$y_1[n]=x_1[n]*_c h_1[n]=\sum_{k=0}^{P-1}x_1[k]h_1[((n-k))_P]$$

For example,

$$y[2]=x[0]h[2]+x[1]h[1]+x[2]h[0]+x[3]h[-1]+x[4]h[-2]+\cdots\cdots$$

$$y_1[2]=x_1[0]h_1[2]+x_1[1]h_1[1]+x_1[2]h_1[0]+x_1[3]h_1[P-1]+x_1[4]h_1[P-2]$$
$$+\cdots\cdots+x[P-1]h[3]$$

The condition where <u>the circular convolution</u> is equal to <u>the linear convolution</u>:

(i) $x[n]=0 \quad for\ n<0 \ \ or \ \ n\geq N$

$$\boxed{x_1[n]=x[n] \quad for\ 0\leq n<N, \quad x_1[n]=0 \quad for\ N\leq n<P-1}$$

(ii) $h[n]=0 \quad for\ n<0 \ \ or \ \ n\geq M$

$$\boxed{h_1[n]=h[n] \quad for\ 0\leq n<M, \quad h_1[n]=0 \quad for\ M\leq n<P-1}$$

(iii) $P\geq N+M-1$

The condition where the circular convolution is equal to the linear convolution:

(i) $x[n]=0 \quad \mathit{for}\ n<0\ \ \mathit{or}\ \ n\geq N$

$$\boxed{x_1[n]=x[n] \quad \mathit{for}\ 0\leq n<N, \quad x_1[n]=0 \quad \mathit{for}\ N\leq n<P-1}$$

(ii) $h[n]=0 \quad \mathit{for}\ n<0\ \ \mathit{or}\ \ n\geq M$

$$\boxed{h_1[n]=h[n] \quad \mathit{for}\ 0\leq n<M, \quad h_1[n]=0 \quad \mathit{for}\ M\leq n<P-1}$$

(iii) $P\geq N+M-1$

(Proof): $y_1[n]=\sum_{k=0}^{N-1} x_1[k]h_1[((n-k))_P]$

$$y_1[n]=x_1[0]h_1[n]+x_1[1]h_1[n-1]+\cdots+x_1[n]h_1[0]\cancel{+x_1[n+1]h_1[P-1]+}$$

$$\cancel{x_1[n+2]h_1[P-2]+\cdots}+x_1[P+n+1-M]h_1[M-1]+\cdots+x_1[P-1]h_1[n+1]$$

$$=x_1[0]h[n]+x_1[1]h[n-1]+\cdots+x_1[n]h[0]$$

$$\cancel{+x_1[P+n+1-M]h[M-1]+\cdots+x_1[P-1]h[n+1]}$$

$$=x[0]h[n]+x[1]h[n-1]+\cdots+x[n]h[0]=y[n]$$

(Since $P+n+1-M\geq P+1-M\geq N$)

$$y[n] = x[n] * h[n] = \sum_k x[n-k]\,h[k]$$

Linear Convolution 有幾種 Cases

Case A: Both $x[n]$ and $h[n]$ have infinite lengths. (impossible)

Case B: Both $x[n]$ and $h[n]$ have finite lengths.

Case C: $x[n]$ has infinite length but $h[n]$ has finite length.

Case D: $x[n]$ has finite length but $h[n]$ has infinite length.

We focus on Case B.

Case C and Case D can also be computed.

Case B: Both $x[n]$ and $h[n]$ have finite lengths.

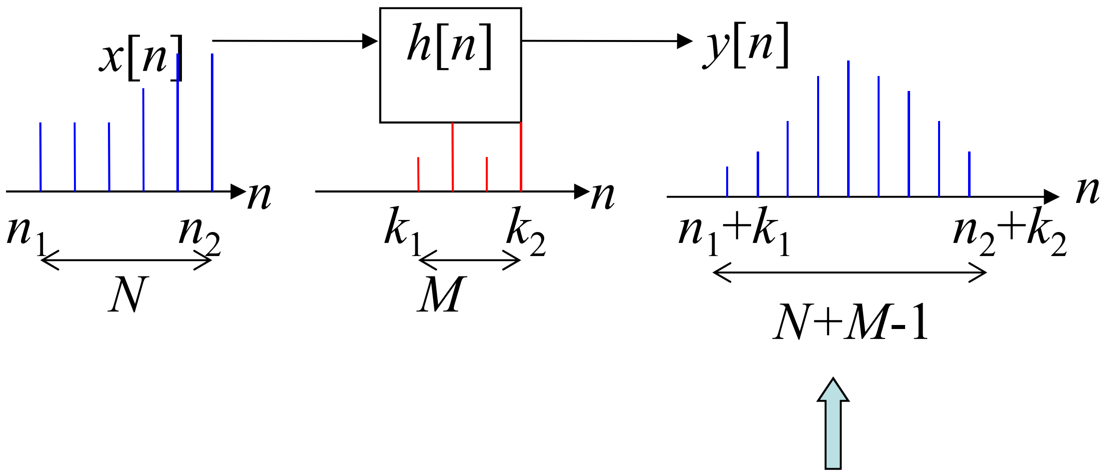
$x[n]$ 的範圍為 $n \in [n_1, n_2]$，大小為 $N = n_2 - n_1 + 1$

$h[n]$ 的範圍為 $n \in [k_1, k_2]$，大小為 $K = k_2 - k_1 + 1$

(n_1, k_1 can be nonzero)

$$y[n] = x[n] * h[n] = \sum_{k=k_1}^{k_2} x[n-k]h[k]$$

$y[n]$ 的範圍?



Convolution output 的範圍以及點數，
是學信號處理的人必需了解的常識

FFT implementation for Case B

$x_1[n] = x[n+n_1]$ for $n = 0, 1, 2, \ldots, N-1$

$x_1[n] = 0$ for $n = N, N+1, \ldots, P-1$ $\quad P \geq N + M - 1$

$h_1[n] = h[n+k_1]$ for $n = 0, 1, 2, \ldots, M-1$

$h_1[n] = 0$ for $n = M, M+1, \ldots, P-1$

$$y_1[n] = IFFT_P\left(FFT_P\{x_1[n]\}\,FFT_P\{h_1[n]\}\right)$$

$y[n] = y_1[n - n_1 - k_1]$ for $n = n_1+k_1,\ n_1+k_1+1,\ n_1+k_1+2, \ldots\ldots, k_2 + n_2$

i.e., $n - n_1 - k_1 = 0, 1, \cdots\cdots, N + M - 2$

取 output 的前面 $N+M-1$ 個點

Case C: $x[n]$ has finite length but $h[n]$ has infinite length

$x[n]$ 的範圍為 $n \in [n_1, n_2]$，範圍大小為 $N = n_2 - n_1 + 1$

$h[n]$ 無限長

$y[n] = \sum_k x[n-k]h[k]$　　$y[n]$ 每一點都有值 (範圍無限大）

但我們只想求出 $y[n]$ 的其中一段

希望算出的 $y[n]$ 的範圍為 $n \in [m_1, m_2]$，範圍大小為 $M = m_2 - m_1 + 1$

$h[n]$ 的範圍？

要用多少點的 FFT？

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[n-k]h[k]$$

改寫成 $y[n] = x[n] * h[n] = \sum_{s=n_1}^{n_2} x[s]h[n-s]$

$$y[n] = x[n_1]h[n-n_1] + x[n_1+1]h[n-n_1-1] + x[n_1+2]h[n-n_1-2]$$
$$+\cdots\cdots + x[n_2]h[n-n_2]$$

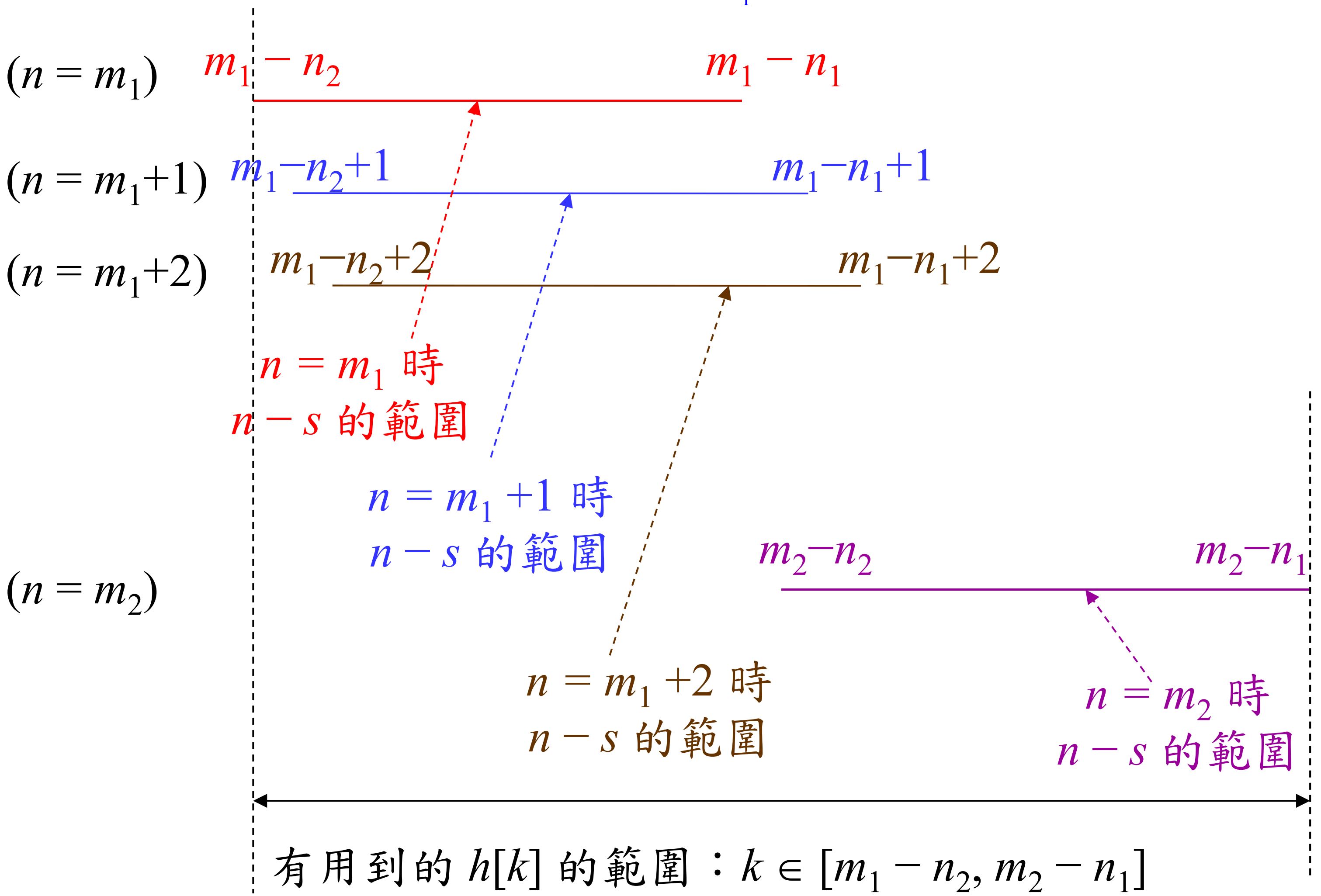
當 $n = m_1$

$$y[m_1] = x[n_1]h[m_1-n_1] + x[n_1+1]h[m_1-n_1-1] + x[n_1+2]h[m_1-n_1-2]$$
$$+\cdots\cdots + x[n_2]h[m_1-n_2]$$

當 $n = m_2$

$$y[m_2] = x[n_1]h[m_2-n_1] + x[n_1+1]h[m_2-n_1-1] + x[n_1+2]h[m_2-n_1-2]$$
$$+\cdots\cdots + x[n_2]h[m_2-n_2]$$

此圖為 $n-s$ 範圍示意圖　　$y[n]=\sum_{s=n_1}^{n_2} x[s]h[n-s]$



所以有用到的 $h[k]$ 的範圍是 $k \in [m_1 - n_2 , m_2 - n_1]$

範圍大小為 $m_2 - n_1 - m_1 + n_2 + 1 = N + M - 1$

FFT implementation for Case 3

$x_1[n] = x[n + n_1]$ for $n = 0, 1, 2, \ldots, N-1$

$x_1[n] = 0$ for $n = N, N+1, N+2, \ldots\ldots, P-1$ $P \geq N + M - 1$

$h_1[n] = h[n + m_1 - n_2]$ for $n = 0, 1, 2, \ldots, L-1$

$$y_1[n] = IFFT_P\left(FFT_P\{x_1[n]\} FFT_P\{h_1[n]\}\right)$$

$y[n] = y_1[n - m_1 + N - 1]$ for $n = m_1, m_1+1, m_1+2, \ldots, m_2$

$$n - m_1 + N - 1 = N - 1, N, \cdots\cdots, N + M - 2$$

注意：$y[n]$ 只選 $y_1[n]$ 的第 N 個點到第 $N+M-1$ 個點

⊙ 11-D Relations between the Signal Length and the Convolution Algorithm

Suppose that

$x[n]$: input, $h[n]$: the impulse response of the filter

length($x[n]$) = N, length($h[n]$) = M (Both of them have finite lengths)

$y[n] = x[n]h[0] + x[n-1]h[1] + x[n-2]h[2] + \cdots + x[n-M+1]h[M-1]$

We want to compute

NM multiplications, 3MN real multiplications

$$y[n] = \sum_{m=0}^{M-1} \underbrace{x[n-m]}_{N}\,\underbrace{h[m]}_{M}, \qquad y[n] = x[n] * h[n].$$

The above convolution needs the P-point DFT, $P \geq M + N - 1$.

complexity: $O(P \log_2 P)$

[Case 1]: When *M* is a very small integer:

Directly computing

Number of multiplications for directly computing: $N \times M$

- Number of real multiplications for directly computing:

$3N \times M$ If $x[n]$, $h[n]$ are real: $N \times M$

- When using $y[n] = IFFT_P\left(FFT_P\{x[n]\}FFT_P\{h[n]\}\right)$

Number of real multiplications

$$2MUL_P + 3P$$

MUL_P: the number of multiplications for the *P*-point DFT

When $3N \times M \leq 2MUL_P + 3P$,

it is proper to do directly computing instead of applying the DFT.

Example: $N = 126$, $M = 3$, (difference, edge detection)

$(3/2)\log_2 N = 10.4659$

When compute the number of real multiplications explicitly,

using direct implementation: $3N \times M = 1134$,

using the 128-point DFT:

using the 144-point DFT: $P \geq M+N-1 = 128$

$2\,MUL_{144} + 3 \times 144 = 2 \times 436 + 432 = 1304$

If $N=126$, $M=4$ direct: $126 \times 4 \times 3 = 1512$ but if x, h are real $126 \times 4 = 504$

DFT: $P \geq 126+4-1 = 129$, set $P = 144$

$2\,MUL_{144} + 3 \times 144 = 1304$

Although in usual "directly computing" is not a good idea for convolution implementation, in the cases where

(a) M is small

(b) The filter has some symmetric relation

using the directly computing method may be efficient for convolution implementation.

$y[n] = \sum_m x[n-m]\, h[m]$

Example: edge detection

$h[n] = [-0.1,\ -0.3,\ -0.6,\ 0,\ 0.6,\ 0.3,\ 0.1]$ for $n = -3 \sim 3$

$y[n] = 0.1x[n-3] + 0.3x[n-2] + 0.6x[n-1] + 0x[n] - 0.6x[n+1] - 0.3x[n+2]$
$- 0.1x[n+3]$
$= 0.1(x[n-3]-x[n+3]) + 0.3(x[n-2]-x[n+2]) + 0.6(x[n-1]-x[n+1])$
$= 0.1\big((x[n-3]-x[n+3]) + 0.3(x[n-2]-x[n+2] + 2(x[n-1]-x[n+1])\big)$

if $x[n]$ is real, $2N$ MULs

Example: smooth filter

$h[n] = [0.1, \; 0.2, \; 0.4, \; 0.2, \; 0.1]$ for $n = -2 \sim 2$

$y[n] = 0.1x[n-2] + 0.2x[n-1] + 0.4x[n] + 0.2x[n+1] + 0.1x[n+2]$

$= 0.1(x[n-2] + x[n+2] + 2(x[n-1] + x[n+1]) + 4x[n])$

if $x[n]$ is real and length(x) = N

N MULs

[Case 2]: When M is not a very small integer but much less than N ($N >> M$):

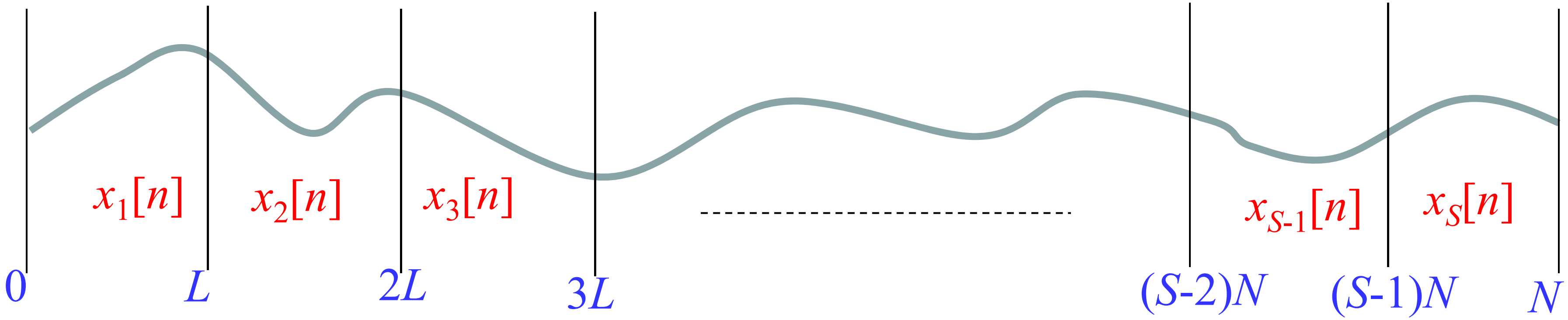
It is proper to divide the input $x[n]$ into several parts:

Each part has the size of L ($L > M$). This method is **sectioned convolution**.

$x[n]$ (n = 0, 1, …., N–1) $\rightarrow$ $x_1[n]$, $x_2[n]$, $x_3[n]$, …….., $x_S[n]$

無條件進位

$x[n]$ $S = \lceil N/L \rceil$, $\lceil\ \rceil$ means rounding toward infinite



Section 1 $x_1[n] = x[n]$ for n = 0, 1, 2, …., L –1,

Section 2 $x_2[n] = x[n + L]$ for n = 0, 1, 2, …., L –1,

:

Section s $x_s[n] = x[n + (s\text{-}1)L]$ for n = 0, 1, 2, …., L –1,

s = 1, 2, 3, …., S

$$x[n] = \sum_{s=1}^{S} x_s[n-(s-1)L]$$

$$y[n] = x[n] * h[n] = \sum_{s=1}^{S} x_s[n-(s-1)L] * h[n] = \sum_{s=1}^{S} y_s[n-(s-1)L]$$

where $y_s[n] = x_s[n] * h[n] = \sum_{m=0}^{M-1} \underbrace{x_s[n-m]}_{\text{length} = L}\, \underbrace{h[m]}_{\text{length} = M}$

It should perform the P-point FFTs $2S$ times. Why?

$P \geq L+M-1$

Detail of Implementation

Suppose that the P-point DFT is applied for each section

$$x[n] = 0 \quad \text{when } n < 0 \text{ and } n \geq N$$

$P \geq L+M-1 \quad L \leq P-M+1$

(1) First, determine $L = P - M + 1$

(2) $x_s[n] = x[(s-1)L+n]$ for $n = 0, 1, 2, \ldots, L-1$, $s = 1, 2, 3, \ldots, S$

$x_s[n] = 0$ for $n = L, L+1, \ldots, P-1$ $\quad S = \lceil N/L \rceil$

$h_1[n] = h[n]$ for $n = 0, 1, 2, \ldots, M-1$,

$h_1[n] = 0$ for $n = M, M+1, \ldots, P-1$

(3) Then calculate

$$y_s[n] = IDFT_P\left\{DFT_P\left(x_s[n]\right)\underbrace{DFT_P\left(h_1[n]\right)}_{\text{pre-calculation}}\right\} \rightarrow 2MUL_P + 3P$$

(4) Then, apply "overlapped addition"

$$y[n] = \sum_{s=1}^{S} y_s[n-(s-1)L]$$

$\star\ \Theta(N)$

運算量：$2S \times \mathrm{MUL}_P + 3S \times P$ $\quad S \approx N/L, \quad P \approx L+M-1$

MUL_P: the number of multiplications for the P-point DFT

運算量：$2S \times \boxed{[(3P/2)\log_2 P]} + 3S \times P \qquad S \approx N/L, \quad P \approx L+M-1$

(理論值)

$$\approx \frac{N}{L} 3(L+M-1)\left[\log_2(L+M-1)+1\right]$$

(linear with N)

何時為 optimal? $\rightarrow \dfrac{\partial 運算量}{\partial L} = 0$

$$N\frac{L-(L+M-1)}{L^2}\left[\log_2(L+M-1)+1\right] + N\frac{L+M-1}{L}\frac{1}{(L+M-1)\log 2} = 0$$

$$L = (M-1)\left[\log(L+M-1)+\log 2\right]$$

In practice, a computer program is applied to determine the optimal L.

How do we estimate the optimal L?

(1) Find L_0 to minimize $\dfrac{N}{L}3(L+M-1)\left[\log_2(L+M-1)+1\right]$

(2) Estimate P_0 from $P_0 = L_0+M-1$. Then several values of P around P_0 to make MUL_P smaller

(3) Calculate L, S, and the number of real multiplications for each possible P to find the optimal P and L.

(1)

Ex: $N=44100$, $M=21$

$L0=111$, $P_0 = L_0+M-1=131$

optimal

$L=124$, $P=144$

(i) $P=144$ $\mathrm{MUL}_P=436$

$L=P-M+1=124$

$S=\dfrac{N}{L}=356$

$2S\,\mathrm{MUL}_P+3SP=4\cancel{64224}$ 464224

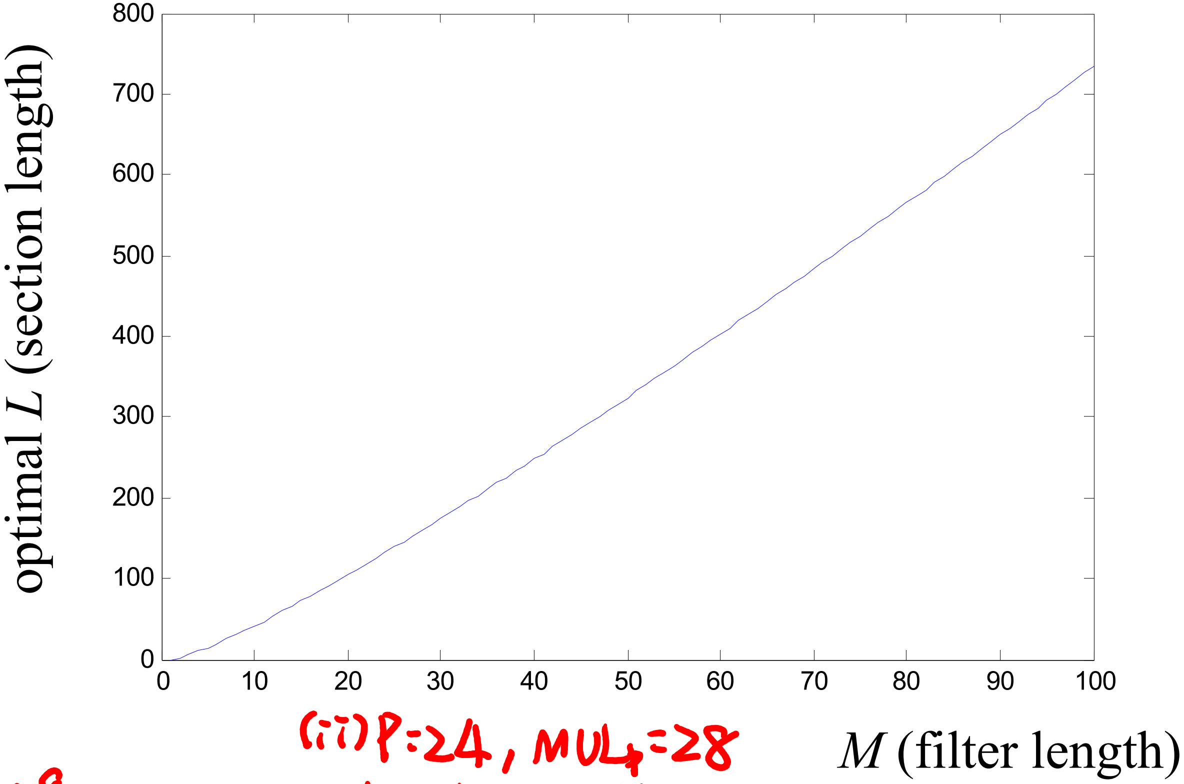
(ii) $P=120$ $\mathrm{MUL}_P=380$

$L=P-M+1=100$, $S=\dfrac{N}{L}=441$

$2S\,\mathrm{MUL}_P+3SP=493920$

(iii) $P=168$, $\mathrm{MUL}_P=580$

$L=P-M+1=148$, $S=\dfrac{N}{L}$, $2S\,\mathrm{MUL}_P+3SP=495872$



Ex 2:

$N=4000$, $M=8$

$L_0=30$ $P_0=L_0+M-1=37$

(i) $P=36$, $MUL_p=64$

$L=P-M+1=29$, $S=\frac{N}{L}=138$

$2SMUL_p+3SP=32568$

(ii) $P=24$, $MUL_p=28$

$L=17$, $S=\frac{N}{L}=236$

$2SMUL_p+3SP=30208$

(iii) $P=16$ ... 39160

(iv) $P=48$, $MUL_p=92$

?

注意：

(1) Optimal section length is independent to N

(2) If M is a fixed constant, then the complexity is linear with N, i.e.,

$$O(N)$$

比較：使用原本方法時， complexity = $O((N+M\text{-}1)\log_2(N+M\text{-}1))$

(3) 實際上，需要考量 P-point FFT 的乘法量必需不多

$$P = L+M-1$$

例如，根據 page 444 的方法，算出當 $M = 10$ 時， $L = 41.5439$ 為 optimal

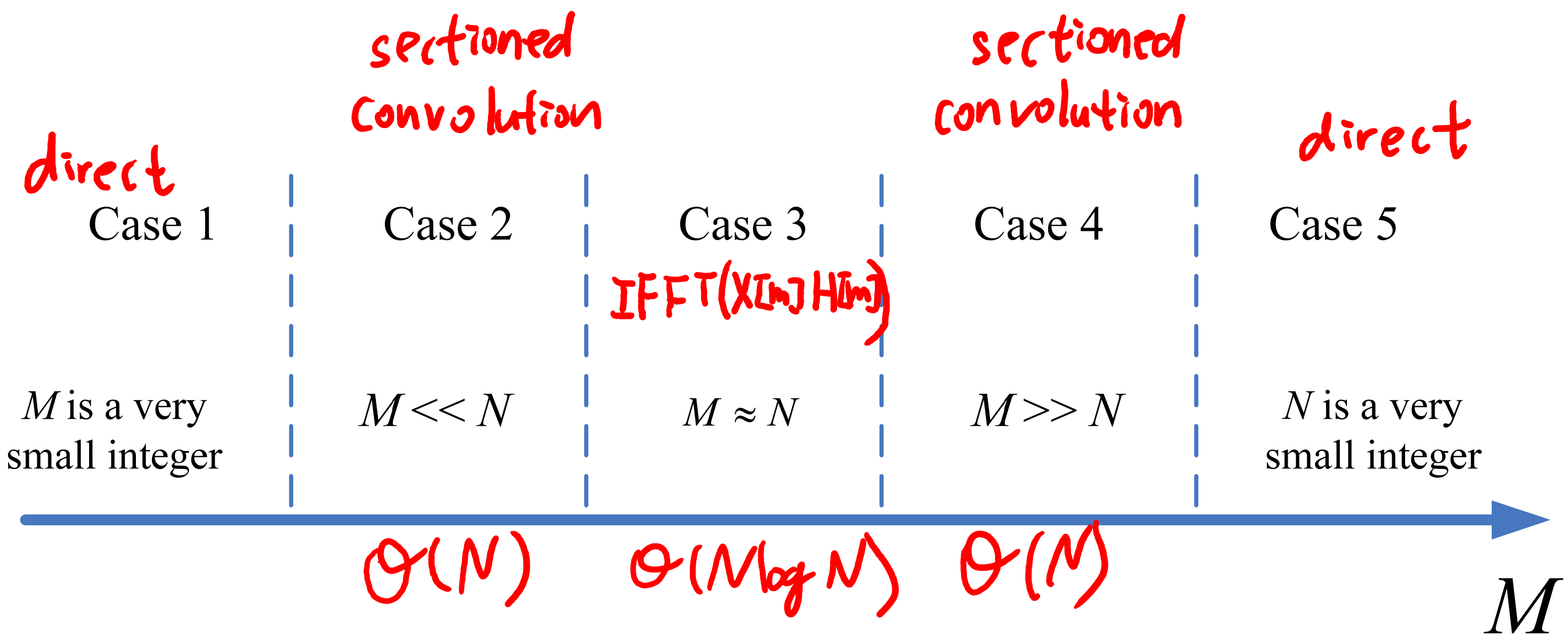
但實際上，應該選 $L = 39$，因為此時 $P = L+M-1 = 48$ 點的 DFT 有較少的乘法量

[Case 3]: When M has the same order as N

[Case 4]: When M is much larger than N

[Case 5]: When N is a very small integer

$$y[n] = x[n] * h[n], \quad \text{length}(x[n]) = N, \quad \text{length}(h[n]) = M$$



- **Sectioned Convolution for the Condition where One Sequence is Finite and the Other One is Infinite**

$$y[m] = \sum_n x[n]h[m-n]$$

$x[n] \neq 0$ for $n_1 \leq n \leq n_2$, length of $x[n] = N = n_2 - n_1 + 1$,

length of $h[n]$ is infinite,
and we want to calculate $y[m]$ for $m_1 \leq m \leq m_2$, $M = m_2 - m_1 + 1$.

Suppose that $M << N$.

In this case, we can try to partition $x[n]$ into several sections.

section 1: $x_1[n] = x[n]$ for $n = n_1 \sim n_1 + L - 1$, $x_1[n] = 0$ otherwise,
section 2: $x_2[n] = x[n]$ for $n = n_1 + L \sim n_1 + 2L - 1$, $x_2[n] = 0$ otherwise,

$\vdots$

section q: $x_q[n] = x[n]$ for $n = n_1 + (q-1)L \sim n_1 + qL - 1$, $x_q[n] = 0$ otherwise,

$\vdots$

Then we perform the convolution of $x_q[n] * h[n]$ for each of the sections by the method on pages 433-435.

(Since the length of $x_q[n]$ is L, it requires the P-point DFT,

$$P \geq L+M-1.$$

Its complexity and the optimal section length can also be determined by the formulas on page 444.

◉ 11-E Recursive Method for Convolution Implementation

$$y[n]=\sum_{m=0}^{N-1}x[n-m]a\cdot b^{m}=x[n]*a\cdot b^{n}u[n]$$

$u[n]$: unit step function

$$Y(z)=X(z)\frac{a}{1-bz^{-1}}$$

$$(1-bz^{-1})Y(z)=aX(z)$$

$$y[n]=by[n-1]+ax[n]$$

Only two multiplications required for calculating each output.

$$y[n] = x[n] * h[n]$$

$$h[n] = 0.25 \cdot 0.6^{|n|}$$

$$\begin{aligned} H(z) &= \frac{0.25}{1-0.6z^{-1}} + \frac{0.25}{1-0.6z} - 0.25 \\ &= 0.25 \frac{1}{\frac{1.36}{0.64} - \frac{0.6}{0.64}\left(z^{-1} + z\right)} \end{aligned}$$

12. Fast Algorithm 的補充

◉ 12-A Discrete Fourier Transform for Real Inputs

DFT: $$F[m] = \sum_{n=0}^{N-1} f[n] e^{-j\frac{2\pi}{N}mn}$$

當 $f[n]$ 為 real 時， $F[m] = F^*[N-m]$ *: conjugation

$$F^*(N-m) = \sum_{n=0}^{N-1} f^*[n]\, e^{j\frac{2\pi}{N}(N-m)n}$$

$$= \sum_{n=0}^{N-1} f^*[n]\, e^{-j\frac{2\pi}{N}mn}\, \cancel{e^{j2\pi n}}$$

$$= F[m]$$

若我們要對兩個 real sequences $f_1[n]$ ，$f_2[n]$ 做 DFTs

Step 1: $f_3[n] = f_1[n] + j f_2[n]$

Step 2: $F_3[m] = \mathrm{DFT}\{f_3[n]\}$

Step 3: $F_1[m] = \dfrac{F_3[m] + F_3^*[N-m]}{2}$ $\qquad F_2[m] = \dfrac{F_3[m] - F_3^*[N-m]}{2j}$

只需一個 DFT

compute 2 DFTs for real inputs
by only one DFT

證明：由於 DFT 是一個 linear operation

$$F_3[m] = F_1[m] + jF_2[m]$$

又 $\quad F_1[m] = F_1^*[N-m] \qquad F_2[m] = F_2^*[N-m]$

$$F_3[m] + F_3^*[N-m] = F_1[m] + \cancel{jF_2[m]} + F_1^*[N-m] - \cancel{jF_2^*[N-m]}$$
$$= 2F_1[m]$$
$$F_3[m] - F_3^*[N-m] = j2F_2[m]$$

同理，當兩個 inputs 為

(1) pure imaginary

(2) one is real and another one is pure imaginary

時，也可以用同樣的方法將運算量減半

- 若 input sequence 為 even $f[n] = f[N-n]$，
  則 DFT output 也為 even $F[n] = F[N-n]$
- 若 input sequence 為 odd $f[n] = -f[N-n]$，
  則 DFT output 也為 odd $F[n] = -F[N-n]$

若有四個 input sequences，二個為 even and real，二個為 odd and real，
則乘法量可減為 1/4

[Corollary 1]

If we have known that the IDFTs of $F_1[m]$ and $F_2[m]$ are real, then the IDFTs of $F_1[m]$ and $F_2[m]$ can be implemented using only one IDFT:

(Step 1) $F_3[m] = F_1[m] + j\,F_2[m]$

(Step 2) $f_3[n] = \mathrm{IDFT}\{F_3[m]\}$

(Step 3) $f_1[n] = \mathcal{Re}\{f_3[n]\},\; f_2[n] = \mathcal{Im}\{f_2[n]\},$

[Corollary 2]

When $x[n]$ and $h[n]$ are both real, the computation loading of the convolution (or the sectioned convolution) of $x[n]$ and $h[n]$ can be halved.

◉ 12-B Converting into Convolution

一般的 linear operation:

$$z[m]=\sum_{n=0}^{N-1}x[n]k[m,n]$$ (習慣上，把 $k[m, n]$ 稱作 "kernel")

$n = 0, 1, \ldots., N-1, \quad m = 0, 1, \ldots., M-1$

可以用矩陣 (matrix) 來表示

運算量為 MN

若為 linear time-invariant operation:

$$z[m]=\sum_{n=0}^{N-1}x[n]h[m-n]$$ $k[m, n] = h[m - n]$ (dependent on m, n 之間的差)

$n = 0, 1, \ldots., N-1, \quad m = 0, 1, \ldots., M-1$

$m - n$ 的範圍： 從 $1-N$ 到 $M-1$， 全長 $M + N - 1$

運算量為 $L\log_2 L$, $\quad L \geq M+2N-2$

大致上，變成 convolution 後 總是可以節省運算量

例子A $z[m]=\sum_{n=0}^{N-1}x[n]2^{mn}$

可以改寫為

$$z[m]=2^{\frac{m^2}{2}}\sum_{n=0}^{N-1}\left\{x[n]2^{\frac{n^2}{2}}\right\}2^{\frac{-(n-m)^2}{2}}$$

convolution

運算量為 $M+N+L\log_2 L$

例子B Linear Canonical Transform

$$y(k)=A\int_{-\infty}^{\infty}e^{j\alpha k^2+j\beta kt+j\gamma t^2}x(t)\,dt$$

$$y(k)=A\int_{-\infty}^{\infty}e^{j(\alpha+\beta/2)k^2-j\frac{\beta}{2}(k-t)^2+j(\gamma+\beta/2)t^2}x(t)\,dt$$

$$y(k)=Ae^{j(\alpha+\beta/2)k^2}\int_{-\infty}^{\infty}e^{-j\frac{\beta}{2}(k-t)^2}\left[e^{j(\gamma+\beta/2)t^2}x(t)\right]dt$$

General rules：

當$k[m, n]$ 可以拆解成 $A[m] \times B[m-n] \times C[n]$

或 $k[m,n] = \sum_s A_s[m] B_s[m-n] C_s[n]$

即可以使用 convolution

◉ 12-C LUT

LUT (lookup table)

道理和背九九乘法表一樣

記憶體容量夠大時可用的方法

Problem: memory requirement

wasting energy

九九乘法表的例子

附錄十四　創意思考

New ideas 聽起來偉大，但大多是由既有的 ideas 變化而產生

(1) Combination

(2) Analogous

(3) Connection

(4) Generalization

(5) Simplification

(6) Reverse

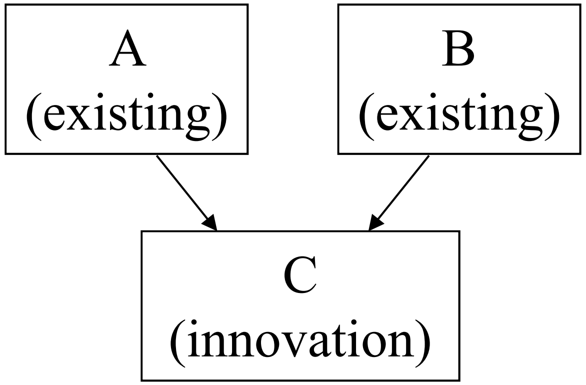
註：感謝已過逝的李茂輝教授，他開的課「創造發明工程」，
讓我一生受用無窮

(7) Key Factor Analysis

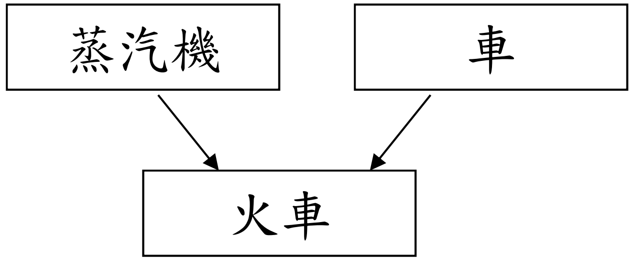
(8) 胡思亂想，純粹意外

(9) 純粹意外

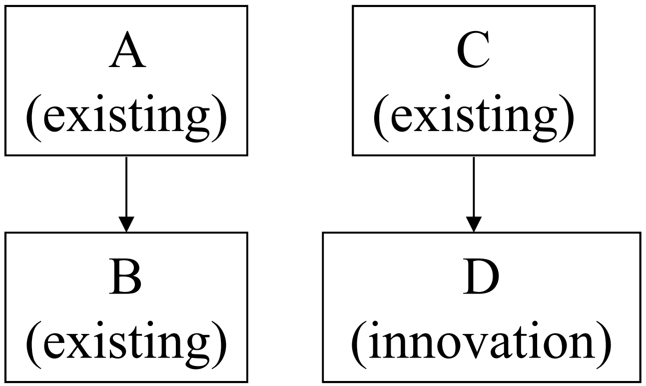
(1) Combination



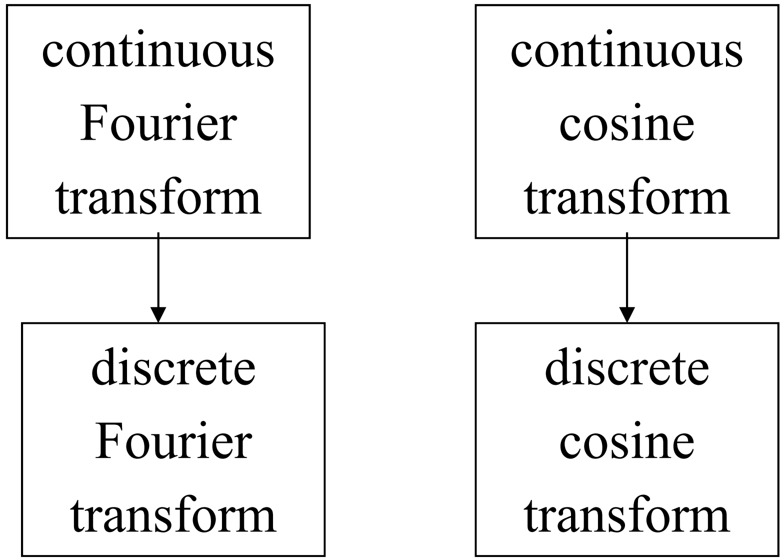
例：



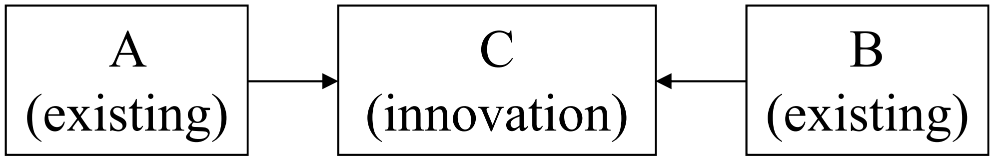
(2) Analog



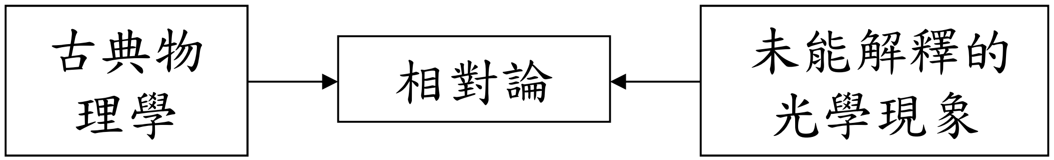
例：



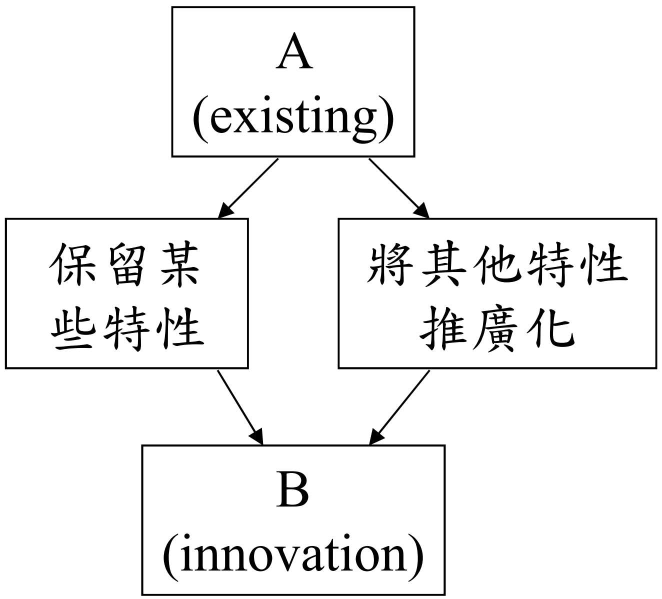
(3) Connection



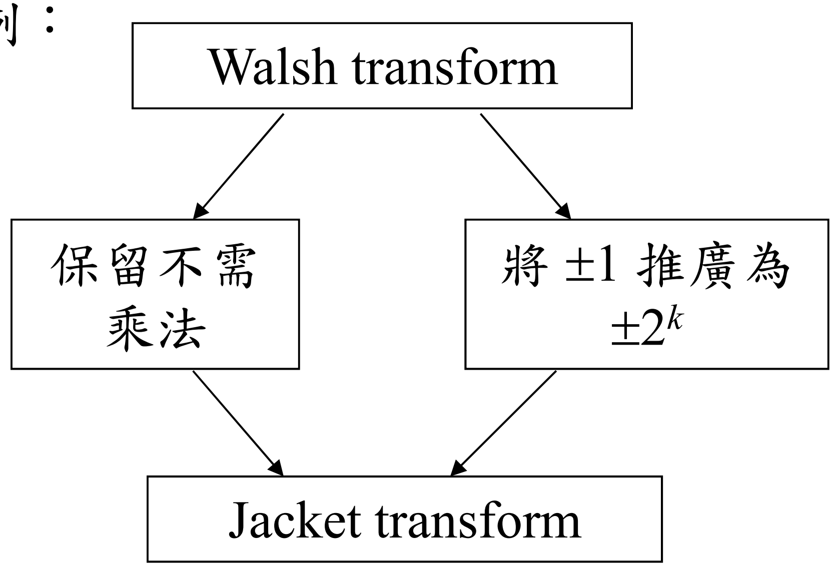
例：



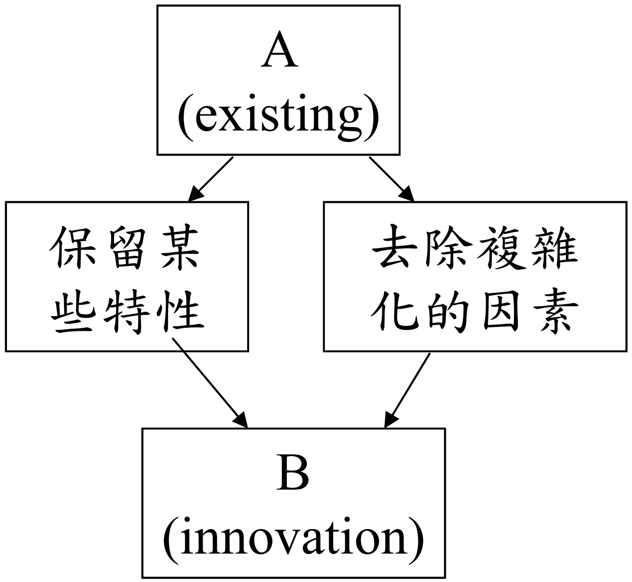
(4) Generalization



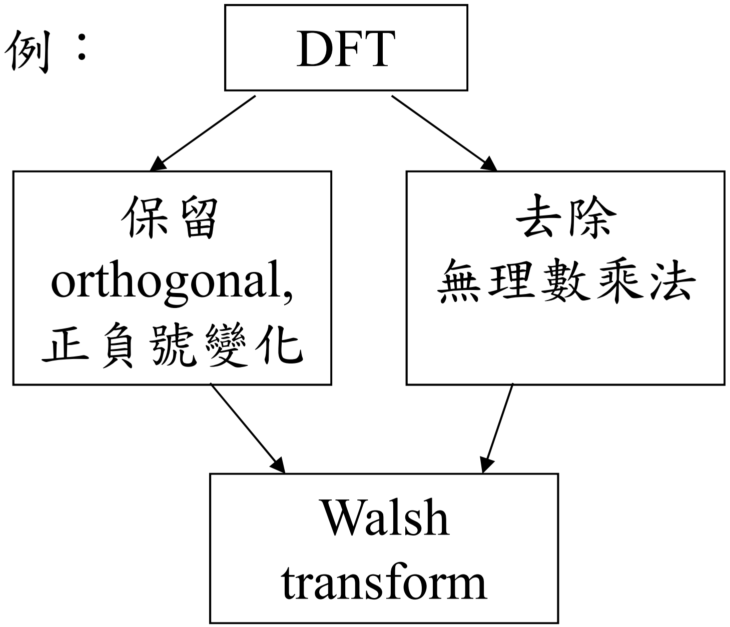
例：



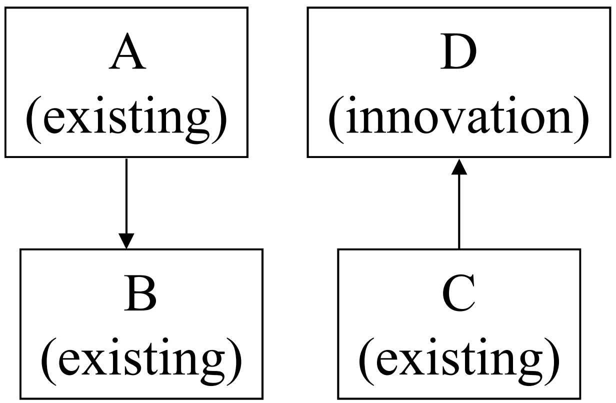
(5) Simplification

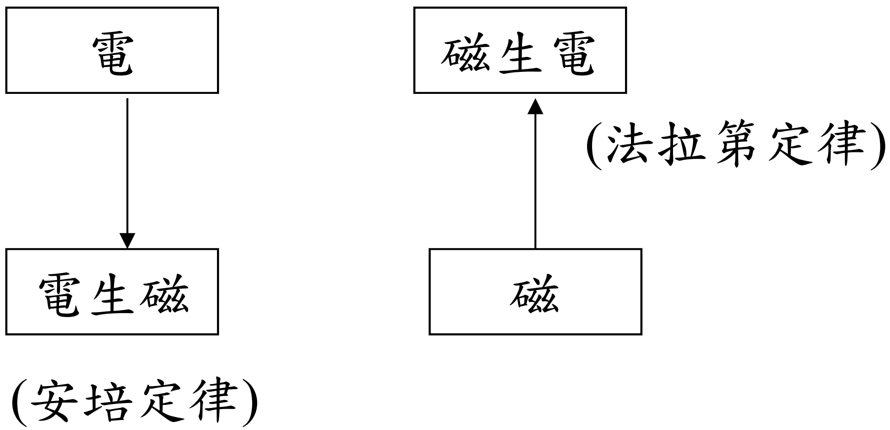


例：



(6) Reverse





個人做研究時，關於發明創造的心得

(1) 研究問題的第一步，往往是先想辦法把問題簡化

如果不將複雜的經濟問題簡化成二維供需圖，經濟學也就無從發展起來

如果不將電子學的問題簡化常小訊號模型，電子電路的許多問題都將難以解決

個人在研究影像處理時，也常常先針對 size 很小，且較不複雜的影像來做處理，成功之後再處理 size 較大且較複雜的影像

問題簡化之後，才比較容易對問題做分析，並提出改良之道

(2) 如果有好點子，趕快用筆記下來，

好點子是很容易稍縱即逝而忘記的。

(3) 練習多畫系統圖

系統圖畫得越多，越容易發現新的點子

(4) 其實，對台大的同學而言，提出 new ideas 並不難，但是要把 ideas 變成有用的、成功的 ideas，不可以缺少分析和解決問題的能力

很少有一個 new idea 一開始就 works well for any case，任何一個成功的創意，都是經由問題的分析，解決一連串的技術上的問題，才產生出來的

(5) 當心情放鬆時，想像力特別強，有助於發現意外的點子。

(6) 就短期而言，技術性的問題固然重要

但是就長期而言，不要因為技術上的困難，而否定了一個偉大的構想

大學以前的教育，是學習前人的智慧結晶

研究所的教育，是訓練創造發明和解決問題的能力